Version Control and GitHub

`git != GitHub`

Git concepts, GitHub workflows, and the command-line habits used in this course

Prepared for Austin Community College
Software Development & Computer Science Department

Alexander Katrompas, PhD
August 24, 2026

Where this tutorial fits

This teaches the technology before the assignment workflow.

Assignment 1a

Environment setup, which may include VM + Ubuntu in later courses, or command-line basics in Programming I.

Assignment 1b

General Git and GitHub tutorial. Learn version control as a system, not only as a submission recipe.

Assignment 2

Simple programming assignment. Learn the full course assignment process.

Assignments 3+

Real programming assignments using the tools and habits already practiced.

Environment → Git literacy → Course procedure → Programming work

Git, repositories, and GitHub are not the same thing

Precision here prevents many later mistakes.

Git

A distributed version-control system. It records the history of a project as commits.

Git repository

A versioned project/history managed by Git. A repository can be local on your computer or remote elsewhere.

GitHub

A service that hosts Git repositories and adds collaboration features such as access control, pull requests, issues, and Actions.

```
Git can work with no GitHub account.
GitHub hosts Git repositories.
```

The four places you must keep distinct

Most beginner Git confusion comes from mixing these up.

Working directory

Files you are editing right now

→

Staging area

Selected changes prepared for the next commit

→

Local history

Commits recorded in your repository

→

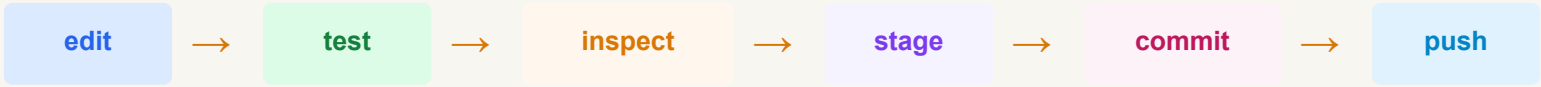
Remote repository

A copy hosted elsewhere, such as GitHub

```
edit          git add          git commit          git push
```

The normal rhythm

Git is not a final upload step. It is part of how you develop.



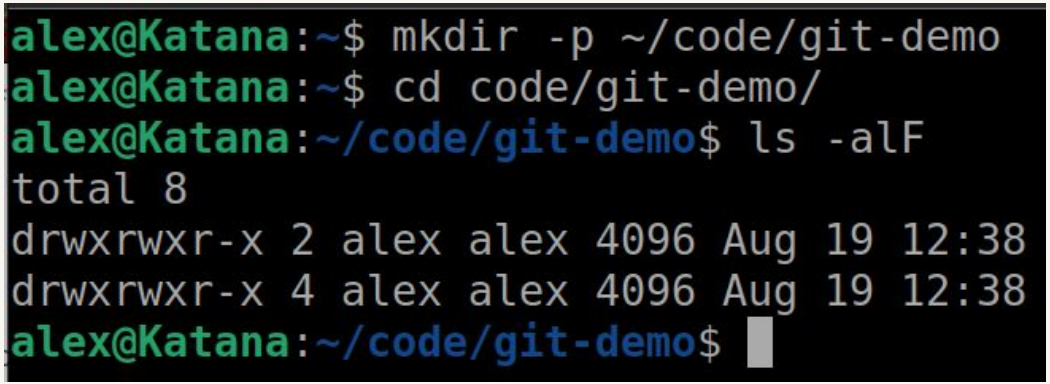
A commit is a coherent checkpoint, not a random save.
You should be able to explain what each commit accomplished.
Push regularly so GitHub has the current work and history.

```
Repeat this cycle throughout the assignment.
Do not write everything first and commit once at the end!
```

Starting with a local project

This tutorial begins local-first to show that Git works before GitHub enters the picture.

- Create a project directory somewhere predictable.
- Put the files for one project in that directory.
- Run Git commands from the root of that project.



OS note

Examples use Bash on Linux. macOS Terminal is similar. Windows students may use PowerShell or Git Bash; command syntax may vary.

One-time Git configuration

Set your identity and the default initial branch name before making repositories.

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --global init.defaultBranch main

git config --global --list
```

Git records author information in commits.

Use the email address associated with your GitHub account when possible.

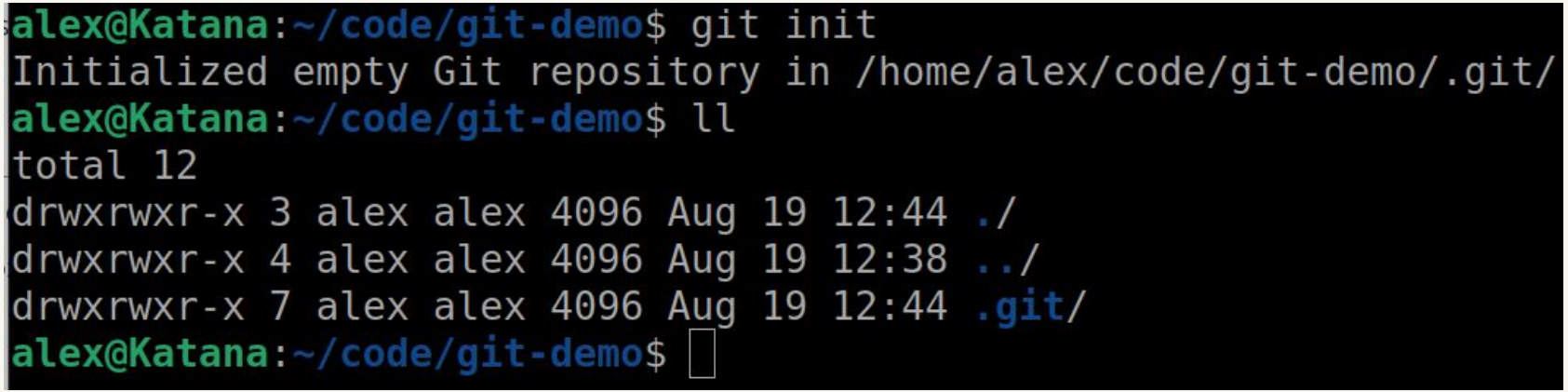
Setting init.defaultBranch avoids the old master/main mismatch.

Do this once per environment

If you use a VM, a lab machine, or a new computer, configure Git there too.

Initialize a local Git repository

git init turns the current project directory into a Git repository.



- **git init** creates a hidden **.git/** directory.
- The **.git/** directory contains Git's internal database and configuration for this project.
- Do not edit or delete **.git/** manually!

Use .gitignore to keep junk out of history

Ignore generated, temporary, local, or sensitive files.

- Not every file in a project belongs in version control.
- Track source code, documentation, tests, and configuration meant to be shared.
- Ignore build outputs, caches, local IDE settings, virtual environments, secrets, and OS clutter.

```
# Python
__pycache__/
*.pyc
.venv/

# C/C++
*.o
*.out
build/

# OS/editor clutter
.DS_Store
.vscode/
```

Course discipline

A shared `.gitignore` normally belongs in the repository because it documents project expectations.

Never commit secrets

Prevention is much easier than recovery.

Do not commit

passwords, API keys, database credentials, private keys, tokens, `.env` files with secrets, production configuration

Assume exposure

If a secret is committed and pushed, treat it as compromised. Rotate the secret instead of merely deleting the file.

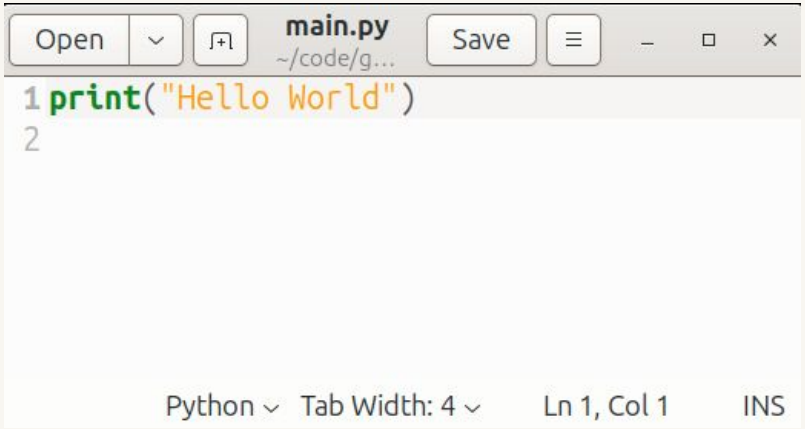
```
# Safer rule:
# if it grants access, it does not belong in Git history.
```

Add a Code File

For demonstration purposes.

Add a code file with any text editor method

use any text editor, IDE, or command line text editor tool




```
alex@Katana:~/code/git-demo$ ll
total 20
drwxrwxr-x 3 alex alex 4096 Aug 19 12:49 ./
drwxrwxr-x 4 alex alex 4096 Aug 19 12:38 ../
drwxrwxr-x 7 alex alex 4096 Aug 19 12:44 .git/
-rw-rw-r-- 1 alex alex  102 Aug 19 12:47 .gitignore
-rw-rw-r-- 1 alex alex   22 Aug 19 12:49 main.py
```

Inspect before you stage or commit

Use Git to see what changed before recording it.

```
git status

# show unstaged changes
git diff

# show staged changes
git diff --staged
```

`git status` tells you what Git sees now.
`git diff` shows changes not yet staged.
`git diff --staged` shows what the next commit would contain.

Habit

Inspect → stage → inspect → commit. This prevents accidental commits.

Stage deliberately with git add

The staging area lets you choose exactly what goes into the next commit.

```
git status

git add main.py .gitignore

git status
git diff --staged
```

Stage only the files that belong to the logical change.
A file can be edited without being staged.
A staged change is ready for the next commit.

Course rule

Avoid broad commands such as `git add .` until you understand exactly what they will stage. Intentional staging is part of professional practice.

Commit the staged snapshot

A commit records a named point in the project history.

```
git commit -m "initial commit hello world with simple .gitignore"

# inspect the result
git log --oneline
```

The commit uses exactly what is staged.

The message should say what changed and why it matters.

After committing, your local repository history has advanced.

Good commits are small, often, and smart

The history should explain how the work was built.

Small

One coherent logical change: one feature step, bug fix, test, or documentation update.

Often

Commit after each completed and tested step. Do not wait until the entire assignment is finished.

Smart

Use messages that describe the change. Avoid “done,” “fix,” “stuff,” or “turning in.”

```
Good:
  Add Celsius-to-Fahrenheit conversion
  Validate temperature input
  Add README usage example

Weak:
  update
  bug fix
  final
```

Read the history you are creating

Git history is useful only if you inspect and understand it.

```
git log --oneline

9f3a2c1 Add README usage example
71d4b0e Validate temperature input
25ce812 Add Fahrenheit conversion
04f6a8a Initial commit
```

Each commit has an identifier called a hash.

The message should let a reader understand the purpose of the commit. History is evidence of process, not just storage.

Instructor perspective

In this course, your repository history is part of the evidence of your development process.

Use diffs to review changes

Diffs show exactly what changed between versions or states.

```
# before staging
git diff

# after staging
git diff --staged

# compare a file with the last commit
git diff HEAD -- main.py
```

What you are looking for

Unexpected file changes, accidental debugging output, temporary notes, formatting noise, or code that should be a separate commit.

Best habit

Read the diff before you commit. This is the Git equivalent of checking your work before submitting it.

A minimal recovery command

Use this carefully; it discards uncommitted changes.

```
# discard unstaged changes in one file
git restore main.py

# unstage a file but keep edits
git restore --staged main.py
```

`git restore` can recover a file from Git's recorded state.

Do not use it casually; it can discard work.

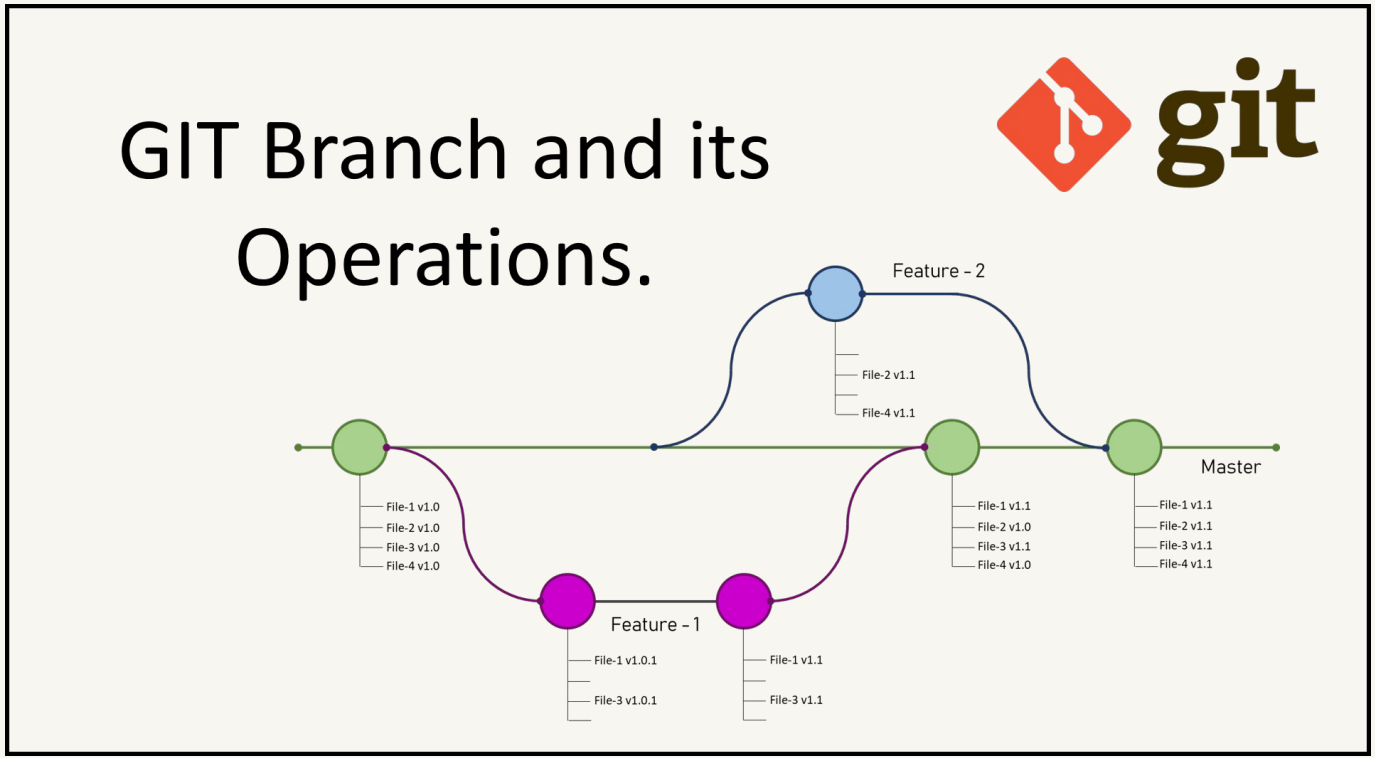
When unsure, copy the file somewhere safe before experimenting.

Beginner rule

If you are not sure what a Git recovery command will do, stop and ask before running it.

Branches: concept only for now

You need to recognize branches, not manage them yet.



Branches: concept only for now

You need to recognize branches, not manage them yet.

```
git branch    # shows branches; * marks the current branch
```

main

The primary line of development used in this course. Your assignment work normally happens here.

feedback

A Classroom 50-managed branch used for the Feedback pull request. Students do not work here.

other branches

Branches can support alternate lines of work, but this introductory tutorial does not teach branching or merging.

A branch is a named line of development inside a repository.
You must recognize which branch you are on.
Unless instructed otherwise, work only on `main`.

Local Git is already useful

At this point, Git works even without GitHub.

- You can track changes over time.
- You can inspect diffs and history.
- You can recover some previous states.
- You can maintain disciplined development checkpoints.

What local Git does not provide

Remote backup, easy multi-machine synchronization, or course/instructor visibility.

```
Local repository: .git/
Remote repository: optional,
until you add one
```

What a remote repository adds

A remote is another repository, usually hosted on a service such as GitHub.

Backup

Your work and history are no longer only on one computer.

Synchronization

You can move between machines or environments more safely.

Collaboration

Other people and tools can see, review, test, and integrate the work.

Create an empty GitHub repository

For the local-first example, the remote must start empty.

- Create a new repository on GitHub.
- Use Private unless you intentionally want the code public.
- For this example, do not initialize the repository with README, .gitignore, or license.
- GitHub will then show commands for connecting an existing local repository.

Why empty?

Your local repository already has its own first commit. Starting with an empty remote avoids competing histories at this stage.

```
Local first workflow needs an
empty remote.
Remote first workflow uses git
clone instead.
```

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).

General

Owner * alexander-katrompas / **Repository name *** git-demo

git-demo is available.

Great repository names are short and memorable. How about vigilant-octo-spork?

Description

demonstration repo for video tutorial

37 / 350 characters

Configuration

Choose visibility * — Choose who can see and commit to this repository — Private

Start with a template — Templates pre-configure your repository with files. — No template

Add README — READMEs can be used as longer descriptions. About READMEs — Off

Add .gitignore — .gitignore tells git which files not to track. About ignoring files — No .gitignore

Add license — Licenses explain how others can use your code. About licenses — No license

Quick setup — if you've done this kind of thing before

HTTPS | SSH | git@github.com:alexander-katrompas/git-demo.git

Get started by creating a new file or uploading an existing file. We recommend every repositc

...or create a new repository on the command line

```
echo "# git-demo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:alexander-katrompas/git-demo.git
git push -u origin main
```

...or push an existing repository from the command line

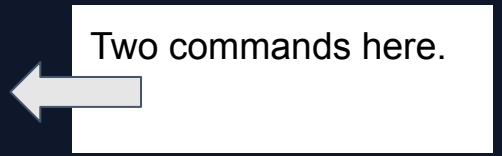
```
git remote add origin git@github.com:alexander-katrompas/git-demo.git
git branch -M main
git push -u origin main
```

Set up SSH authentication for GitHub

SSH lets Git authenticate without typing a GitHub password for every operation.

```
# Generate a modern SSH key
ssh-keygen -t ed25519 -C "your_email@example.com"

# Add it to the SSH agent
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```



Use a passphrase unless your instructor gives a different course-specific instruction.

The private key stays on your machine; the public key is copied to GitHub.

If Ed25519 is unsupported, GitHub documents RSA 4096 as a legacy fallback.

Machine/environment note

A key is associated with the computer or environment where it was generated. A VM may need its own key.

Add the public key to GitHub and test it

Copy the public key only: the file ending in .pub.

```
# Linux example
cat ~/.ssh/id_ed25519.pub

# Then copy the one-line public key to GitHub
# (see next slide for example):
# Settings → SSH and GPG keys → New SSH key

ssh -T git@github.com
```

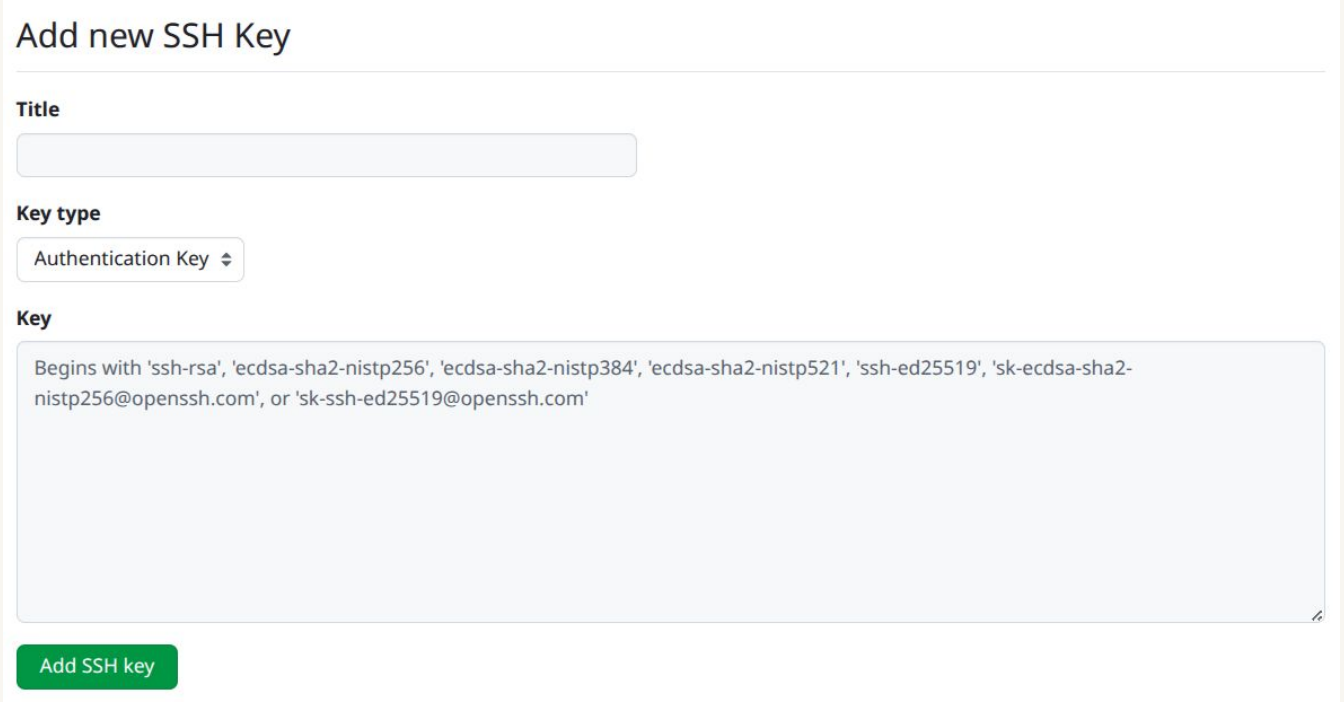
GitHub stores your public key.
Your private key must not be uploaded or shared.

The first SSH connection may ask you to trust GitHub's host key.
A successful test greets your GitHub username.

Windows / macOS hints

GitHub Docs provide platform-specific clipboard commands: pbcopy on macOS, clip/cat variants on Windows, and terminal/text-editor options.

Add the public key to GitHub and test it




```
ssh -T git@github.com
```

Connect the local repository to GitHub

A remote named origin points your local repository at GitHub.

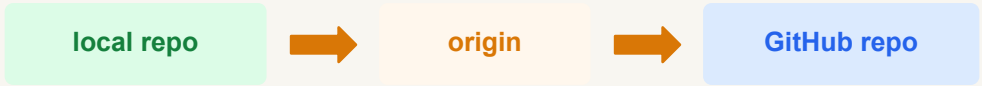
```
git remote add origin git@github.com:USER/REPO.git

# verify the remote
git remote -v
```

`origin` is a conventional name, not magic.

The remote URL can use SSH or HTTPS; this course uses SSH for Git operations.

`git remote -v` shows the configured remote addresses.



Push local commits to the remote

Push moves committed local history to GitHub.

```
# first push: create upstream tracking
git push -u origin main

# later pushes
git push origin main
```

Only committed work can be pushed. A file that is edited but not committed remains local.

The first push with `-u` records the upstream relationship.

```
push: local commits  →  remote repository
```

Verify your work reached GitHub

A successful push should be visible in the browser.

- Open the GitHub repository page.
- Confirm the latest commit message appears.
- Confirm the expected files are present.
- If the browser does not show your latest work, it may still be only local.

```
git status
git log --oneline
git remote -v
```

Submission relevance

For course assignments, your instructor grades the repository state that exists on GitHub when grading begins.

Normal work after setup

Once local and remote exist, this is the recurring cycle.

edit → test → status/diff → add → commit → push → repeat

```
git status
git diff
git add file1 file2
git commit -m "clear message"
git push origin main
```

Never treat GitHub as a ZIP-file drop box

The development history is part of the work. Use commits throughout the process.

Pull remote changes to local

Pull moves remote updates into your local branch.

```
# before starting work in an existing clone
git pull origin main

# then continue locally
edit → test → add → commit → push
```

Use pull when the remote has changes your local repository does not have.
Avoid editing the same work in GitHub and locally without synchronizing.
If Git reports a conflict, stop and resolve carefully before continuing.

```
pull: remote changes  →  local branch
```

Starting from an existing remote repository

This is the path you will use for assignments.

```
cd ~/code/cosc1336

git clone git@github.com:ORG/REPO.git
cd REPO
git status
```

`git clone` creates the local repository directory.

It downloads files and history.
It configures the remote named `origin` automatically.

It checks out the default branch, usually `main`.

Connection to Classroom 50

Classroom 50 creates the GitHub repository first. You then clone it and work locally.

Two starting workflows, one development model

After setup, the daily cycle is the same.

Local first

git init
create .gitignore
git add files
git commit
create empty GitHub repo
git remote add origin ...
git push -u origin main

Remote first

create / accept remote repository
git clone ...
cd repository
edit
test
git add
git commit
git push

Classroom 50 assignments use the remote-first workflow.

Command summary

Know what each command moves or records.

```
git init                start a local repository
git status              inspect current state
git diff                inspect unstaged changes
git diff --staged       inspect staged changes
git add file            stage selected changes
git commit -m "msg"     record staged snapshot
```

```
git log --oneline       inspect history
git branch              see current branch
git remote -v           inspect remotes
git remote add origin   connect local to remote
git push origin main    local → remote
git pull origin main    remote → local
git clone URL           remote → new local clone
```

Core idea

Do not memorize blindly. Understand which place is changing: working directory, staging area, local history, or remote repository.

Final safety rules

These prevent most beginner Git disasters.

- Never edit or delete `.git/` manually.
- Never commit secrets or private keys.
- Stage deliberately; do not blindly add everything.
- Commit small, coherent, tested steps.
- Push regularly so the remote is current.
- Use `main` unless your instructor explicitly tells you otherwise.

```
When unsure:
1. git status
2. read the message carefully
3. stop before running destructive commands
```

WARNING: Never change both remote and local files without syncing them with a pull or push. If you do, you will not be able to pull or push until you resolve the conflict (which may not be easy).

References

Primary documentation used for the technical updates.

Git Book

What is Git? Working with Remotes. Branches in a Nutshell.

Git command docs

git-init, git-add, git-status, git-diff, git-log, git-restore.

GitHub Docs

Generating SSH keys, adding keys to GitHub, working with remotes

Use official documentation when details change.
This presentation teaches general Git and GitHub concepts.
Classroom 50 assignment procedures are covered separately.

```
Primary references: git-scm.com/docs, git-scm.com/book, docs.github.com
```